



Assignment 2

Released: November 13th 2025

Due: November 24th 2025 (via Canvas, before 23:59)

Group number:

Name, student id

Name, student id

Points, grade of 100 points **grade:**

Introduction

The objective of this assignment is to test your understanding of logical programming using Prolog. You are supposed to work on this assignment in **groups of 2 students**.

Programming is something which cannot be learned without actually doing it, and the assignment will therefore ask you to complete several exercises on writing Prolog programs. This assignment covers Prolog search, lists, and recursion. It requires you to have a good understanding of the material covered in the lectures. For the Prolog part it also requires you to read [Learn Prolog Now](#), Chapters 3-6

- **Working in pairs** does not mean to split the exercises and to solve them independently of your group member. We want you to work out the solutions *together*. Each student should understand all the exercises and be able to explain the solution. **This will be tested in the final exam.**
- You are required to have run and test Prolog code on your own machine, which is part of assignment 0, prior to working on this assignment. We use SWI Prolog version 9.2.9-1 for this course (latest stable at time of writing).
- Create **one .pl file** and include your Prolog solutions and written explanation and motivation as comments. Submit the .pl file on Canvas. Note: only one member of the group needs to submit it.

Exercises

Exercise 1 (Campus Navigation Assistant – 15 pts).

You are asked to design a simple navigation program for a university campus. The following knowledge base contains information about travel connections between several campus buildings. The predicates `bike/3` and `walk/3` describe how long it takes to go from one place to another by bike or on foot, respectively.

```
bike(library, cafeteria, 3).
bike(cafeteria, main_gate, 2).
bike(lab_building, library, 4).

walk(main_gate, admin_office, 5).
walk(library, lecture_hall, 6).
walk(lecture_hall, lab_building, 7).
```

- 1 (4 pts.) Define a predicate `route_by_bike/2` such that `route_by_bike(F, T)` succeeds if there exists a direct or indirect connection from F to T by bike.
- 2 (5 pts.) Define a predicate `route/2` such that `route(F, T)` succeeds if it is possible to reach T from F either by bike or by walking (possibly combining both).
- 3 (6 pts.) Extend the predicate to `route/3` such that `route(F, T, M)` succeeds if there is a route from F to T (mixing walking and biking) and M is the total time in minutes. *Hint:* You can sum the minutes of subroutes using arithmetic after the recursive call. You do not need to use accumulators.

Exercise 2 (Blocksworld – 15 pts).

In this exercise, you will continue working with recursive reasoning in the *Blocksworld* domain. Consider the following knowledge base:

```
on(b, a).
on(c, b).
above(X, Y) :- on(X, Y).
above(X, Y) :- on(X, Z), above(Z, Y).
```

The predicate `on(X, Y)` means that block X is directly on top of block Y. The predicate `above(X, Y)` holds when X is located above Y, directly or indirectly.

- 1 (3 pts.) Implement the program and list all pairs (X, Y) for which the query `?- above(X, Y).` succeeds.
- 3 (3 pts.) Define a predicate `atLeastThree(X)` which is true if and only if X is a block standing on top of *at least three* other blocks. You may assume that the block structure is simple (like a straight tower).
- 2 (4 pts.) Define an alternative predicate `atLeastThree_2(X)` that does the same as `atLeastThree(X)`, only now your solution should use the built-in function `setof/3` to collect all blocks below X.
- 4 (5 pts.) A stack of blocks can also be represented as a tower using lists. Consider the following Prolog program:

```

on(a,b).
on(b,c).
on(c,d).
on(d,e).
on(e,table).

tower([X]) :- on(X,table).
tower([X,Y|T]) :- on(X,Y), tower([Y|T]).

```

Write a predicate `exactlyThreeTower(X)` which is true if and only if `X` is a block standing on top of a tower with exactly three other blocks below it (and the lowest block resting on the table). *Hint:* You may use the built-in predicate `length/2`.

Exercise 3 (A small program 10 pts).

Explain in at most three sentences what the below defined predicate `xyz/3` does. *Hint:* Try several example queries, such as `xyz([1,2,3,4],A,B)` or `xyz(L,[a,b],[c,d])`, and use tracing to understand the recursive process and the data flow between arguments.

```

xyz(L,A,B) :- xy(L,[],A,B).
xy([],A,A,[]).
xy([H|T],A,B,[H|R]) :- xy(T,[H|A],B,R).

```

Exercise 4 (Lists – 10 pts).

Write a predicate `balance(A,B,C)` where `A`, `B`, and `C` are lists of integers. The predicate should be true if and only if:

- The concatenation of `B` and `C` (i.e., `append(B,C,BC)`) is a permutation of the elements in `A`; and
- The *sum of the squares* of all integers in `B` equals the sum of the squares of all integers in `C`.

You may assume that all list elements are integers. The variables can be used both as input and output. *Hint:* You may find the built-in predicate `permutation/2` and `append/3` useful.

For example:

```

?- balance([3,4,5], X, Y).
X = [3, 4],
Y = [5];
X = [4, 3],
Y = [5];
X = [5],
Y = [3, 4];
X = [5],
Y = [4, 3].

?- balance([1,2,3], X, Y).
false.

```

Exercise 5 (Recursive predicate – 15 pts).

Define a recursive predicate `midrange(L,M)` which computes the integer midrange `M` of a list of integers `L`, where the midrange is defined as the average of the minimum and maximum elements in the list.

Your predicate should traverse the list only once, that is, it should determine both the minimum and maximum values in a single recursion. You may **not** use built-in predicates such as `min_list/2` or `max_list/2`. Try to rely only on pattern matching and recursion (no special control structures are needed).

Examples:

```
?- midrange([1,4,5,6], M).  
M = 3.  
?- midrange([7,1,6,2], M).  
M = 4.  
?- midrange([5], M).  
M = 5.
```

Hint: Define a helper predicate that carries along the current minimum and maximum values while traversing the list.

Exercise 6 (Music Albums – 35 pts).

In this exercise, you will organize your music albums using Prolog. Information is stored using the predicate `album/6`:

- Title is the album title,
- Artists is a list of artists, each represented as `artist('lastname','firstname')`,
- Genres is a list of genres, each represented as `genre('type')`,
- Tracks is the number of tracks in the album,
- Duration is the total duration in minutes,
- Rating is your personal rating from 1 (worst) to 10 (best).

Here is a sample database:

```
album("Abbey Road",[artist("Lennon","John"),artist("McCartney","Paul")],  
      [genre("rock")],17,47,10).  
  
album("Thriller",[artist("Jackson","Michael")],[genre("pop"),genre("funk")],9,42,9).  
  
album("Random Access Memories",[artist("Daft","Guy-Manuel"),artist("Daft","Thomas")],  
      [genre("electronic")],13,74,8).  
  
album("The Dark Side of the Moon",[artist("Waters","Roger"),artist("Gilmour","David")],  
      [genre("progressive rock")],10,43,10).
```

Assume album titles are unique.

- 1 (5 pts) Write a predicate `allAlbumTitles/1` such that `allAlbumTitles(L)` succeeds if `L` contains all album titles.

- 2 (5 pts) Write a predicate `allAlbumTitlesSorted/1` which returns an alphabetically sorted list of album titles.
- 3 (8 pts) Write a predicate `albumsByGenre/2` such that `albumsByGenre(genre('rock'),L)` returns all album titles in that genre.
- 4 (8 pts) Write a predicate `albumsByRating/2` such that `albumsByRating(AlbumList,SortedTitles)` succeeds if `SortedTitles` contains all album titles in `AlbumList` sorted by rating (highest first).
- 5 (9 pts) Write a predicate `countAlbumsByGenre/3` such that `countAlbumsByGenre(AlbumList,genre('pop'),Count)` succeeds if `Count` is the total number of albums in `AlbumList` belonging to that genre.

END OF ASSIGNMENT